



Objectifs

- Consommer une API REST PHP/MySQL
- Utiliser la bibliothèque Retrofit
- Exploiter des paramètres GET
- Alimenter une RecyclerView
- Gérer un chargement asynchrone avec ProgressBar
- Respecter une architecture Android simple et professionnelle

Qu'est-ce que Retrofit ?

Retrofit est une **bibliothèque Android** (Java/Kotlin) qui permet de communiquer facilement avec une **API REST** (serveur web) en envoyant des requêtes **HTTP (GET, POST, PUT, DELETE)** et en récupérant les réponses sous forme d'**objets Java**.

Contexte

Une **API REST** permet d'accéder à une base de données de DVD. L'application Android doit afficher les DVD d'une catégorie donnée, par l'URL :

```
listByCategory.php?idCat=?
```

GUIDE DE CRÉATION PAS À PAS

1

Créer le projet Android Studio

Créer un projet vide.

1. Ouvrir **Android Studio / New Project**
2. Choisir **Empty Activity**
3. Remplir les champs :

```
Name          : DVDApp
Package name:  com.example.dvd
Language      : Java
Minimum SDK   : API 24
```

4. Cliquer **Finish** et attendre que **Gradle se synchronise**



2 Ajouter les dépendances Gradle

Indique à Android quelles bibliothèques externes on veut utiliser (Retrofit, RecyclerView...).

C'est comme un composer.json en PHP ou un package.json en Node : on liste ce dont on a besoin et Android le télécharge tout seul.

1. Ouvrir **build.gradle** (Module :app) dans **Gradle Scripts**
2. Remplacer le bloc **dependencies { }** par celui ci-dessous
3. Cliquer **Sync Now** pour que Android télécharge automatiquement les bibliothèques

```
dependencies {
    // Android de base
    implementation 'androidx.appcompat:appcompat:1.4.1'
    implementation 'com.google.android.material:material:1.5.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.3'

    // Retrofit (API REST)
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'

    // RecyclerView
    implementation 'androidx.recyclerview:recyclerview:1.3.2'

    // Tests
    testImplementation 'junit:junit:4.13.2'
    androidTestImplementation 'androidx.test.ext:junit:1.1.3'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.4.0'
}
```

Attention : Après modification, toujours cliquer sur « **Sync Now** » en haut de l'écran et attendre la fin de la synchronisation avant de continuer.

3 Configurer le Manifest

Le Manifest c'est la « carte d'identité » de l'application. Android doit savoir à l'avance ce qu'elle va faire et quels écrans elle contient. Déclarer les permissions et les écrans de l'application.

Deux choses importantes à ajouter dans AndroidManifest.xml :

```
<uses-permission android:name="android.permission.INTERNET"/>

<application
    android:usesCleartextTraffic="true">
```

- **INTERNET** : autorise l'appli à faire des requêtes réseau.
- **usesCleartextTraffic="true"** : autorise le HTTP (pas seulement HTTPS), nécessaire pour travailler en local avec WAMP/XAMPP.

**4****Créer les packages**

Organiser le code en **dossiers logiques** nommés **packages**.

Dans **java/com/example/dvd/**, faire **clic droit / New / Package** et créer :

model	→données manipulées
network	→communication avec le serveur
adapter	→affichage de la liste

Architecture complète du projet :

```
app/src/main/  
  java/com/example/dvd/  
    model/  
      DvdModel.java  
      DvdResponse.java  
    network/  
      ApiService.java  
      RetrofitClient.java  
    adapter/  
      DvdRecyclerAdapter.java  
      SerieActivity.java  
  res/layout/  
    activity_serie.xml  
    ligne.xml  
  AndroidManifest.xml
```



5 Créer les modèles de données

Créer les **classes Java** qui représentent ce que l'**API** renvoie.

On crée une **classe Dvd** qui correspond exactement aux colonnes de la **table MySQL**.

Retrofit va automatiquement remplir ces objets à partir du JSON reçu, c'est la **désérialisation**.

DvdModel.java — représente un seul DVD

Le nom de chaque variable doit correspondre exactement au nom du champ JSON retourné par le PHP.

```
package com.example.dvd.model;

public class DvdModel {
    private String titreVideo;        // champ JSON "titreVideo"
    private String nomRealisateur;    // champ JSON "nomRealisateur"
    private String imgVideo;          // champ JSON "imgVideo"

    public String getTitreVideo()      { return titreVideo; }
    public String getNomRealisateur() { return nomRealisateur; }
    public String getImgVideo()        { return imgVideo; }

    // Champ local pour l'ID de la ressource drawable
    private int imgResId;
    public int getImgResId()            { return imgResId; }
    public void setImgResId(int imgResId) { this.imgResId =
imgResId; }
}
```

DvdResponse.java — représente la réponse complète de l'API

```
package com.example.dvd.model;

import java.util.List;

public class DvdResponse {
    private List<DvdModel> table; // clé JSON "table"
    public List<DvdModel> getTable() { return table; }
}
```

Attention :

- Le nom de la variable (table) doit correspondre EXACTEMENT à la clé dans le JSON retourné par le PHP.
- Si le PHP retourne { "dvds": [...] }, il faut écrire `private List<DvdModel> dvds;`

**6****Configurer Retrofit**

Mettre en place la **connexion** avec l'**API PHP**. **Retrofit** qui remplace toute la gestion manuelle des requêtes HTTP (connexion, lecture, parsing JSON...).

ApiService.java — définit les routes disponibles

On définit :

- L'URL appelée et le paramètre
- Le type d'objet en retour.
- Retrofit fait tout le reste automatiquement.

```
package com.example.dvd.network;

import com.example.dvd.model.DvdResponse;
import retrofit2.Call;
import retrofit2.http.GET;
import retrofit2.http.Query;

public interface ApiService {
    @GET("listByCategory.php")
    Call<DvdResponse> getDvdsByCategory(@Query("idCat") int idCat);
    // Génère l'URL : listByCategory.php?idCat=2
}
```

RetrofitClient.java — gère la connexion

C'est un **Singleton** : une seule instance Retrofit est créée pour toute l'application. Pas besoin de recréer la connexion à chaque appel réseau.

```
package com.example.dvd.network;

import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class RetrofitClient {
    private static Retrofit retrofit;
    private static final String BASE_URL =
"http://10.0.2.2/api_dvd/server/";

    public static Retrofit getInstance() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit;
    }
}
```

*L'adresse **10.0.2.2** est l'équivalent de **localhost** depuis l'**émulateur Android**. Depuis un vrai téléphone sur le réseau Wi-Fi, il faudrait mettre l'adresse IP locale de la machine (ex: 192.168.1.x) obtenue avec `ipconfig` dans une fenêtre d'invité de commande `cmd`.*

**7****Créer les layouts XML**

Dessin des écrans de l'application.

activity_serie.xml — l'écran principal

Pendant que les données chargent depuis le réseau, on montre la roue de chargement.

Quand c'est prêt, on cache la roue et on affiche la liste.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- ProgressBar visible au démarrage -->
    <ProgressBar
        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:layout_marginTop="200dp"/>

    <!-- RecyclerView masquée, affichée après chargement -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerDvds"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:visibility="gone"/>

</LinearLayout>
```

ligne.xml — le gabarit d'une ligne de la liste

Ce fichier est répété autant de fois qu'il y a de DVD dans la liste. L'Adapter s'en charge automatiquement.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="8dp">

    <ImageView
        android:id="@+id/image"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:scaleType="centerCrop"
        android:src="@drawable/ic_launcher_background"/>

    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
```



```

        android:layout_marginStart="12dp"
        android:gravity="center_vertical">

        <TextView
            android:id="@+id/titre"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="16sp"
            android:textStyle="bold"/>

        <TextView
            android:id="@+id/realisateur"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="14sp"
            android:textColor="#555555"/>

    </LinearLayout>
</LinearLayout>

```

8 Créer l'Adapter RecyclerView

Fait le lien entre les données (liste de DVD) et l'affichage (les lignes XML).

Les 3 méthodes obligatoires à comprendre :

Méthode	Rôle
onCreateViewHolder	Gonfle (crée) une ligne XML vide
onBindViewHolder	Remplit cette ligne avec les données d'un DVD
getItemCount	Dit combien de lignes il faut afficher

```

package com.example.dvd.adapter;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;
import androidx.recyclerview.widget.RecyclerView;
import com.example.dvd.R;
import com.example.dvd.model.DvdModel;
import java.util.List;

public class DvdRecyclerViewAdapter
    extends RecyclerView.Adapter<DvdRecyclerViewAdapter.ViewHolder>
{
    private List<DvdModel> dvds;
    private Context context;

    public DvdRecyclerViewAdapter(Context context, List<DvdModel> dvds)
    {
        this.context = context;
    }

```



```
        this.dvds    = dvds;
    }

    @Override
    public ViewHolder onCreateViewHolder(ViewGroup parent, int
viewType) {
        View view = LayoutInflater.from(context)
            .inflate(R.layout.ligne, parent, false);
        return new ViewHolder(view);
    }

    @Override
    public void onBindViewHolder(ViewHolder holder, int position) {
        DvdModel dvd = dvds.get(position);
        holder.titre.setText(dvd.getTitreVideo());
        holder.realisateur.setText(dvd.getNomRealisateur());
        int resId = dvd.getImgResId();
        if (resId != 0) {
            holder.image.setImageResource(resId);
        } else {
holder.image.setImageResource(android.R.drawable.ic_menu_gallery);
        }
    }

    @Override
    public int getItemCount() { return dvds.size(); }

    static class ViewHolder extends RecyclerView.ViewHolder {
        TextView titre, realisateur;
        ImageView image;
        ViewHolder(View itemView) {
            super(itemView);
            titre      = itemView.findViewById(R.id.titre);
            realisateur = itemView.findViewById(R.id.realisateur);
            image       = itemView.findViewById(R.id.image);
        }
    }
}
```



9

Créer l'Activity principale

C'est la logique principale — appel de l'API et affichage des résultats.

*Une **Activity** représente le code qui pilote un écran de l'application.*

Flux d'exécution à bien comprendre

```
onCreate()
|
+-- Récupère idCat depuis l'Intent (passé par l'écran précédent)
+-- Affiche la ProgressBar
+-- Lance la requête Retrofit (asynchrone !)
    |
    +-- onResponse() --> masque ProgressBar, affiche la liste
    +-- onFailure()  --> masque ProgressBar, affiche erreur
```

Pourquoi **asynchrone** ? Android interdit les requêtes réseau sur le fil principal (UI thread). Si on bloquait l'écran pendant la requête, l'appli se fige et Android la tue. Retrofit gère cela automatiquement avec enqueue().

```
package com.example.dvd;

import android.os.Bundle;
import android.view.View;
import android.widget.ProgressBar;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import java.util.List;
import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;
import com.example.dvd.adapter.DvdRecyclerAdapter;
import com.example.dvd.model.DvdModel;
import com.example.dvd.model.DvdResponse;
import com.example.dvd.network.ApiService;
import com.example.dvd.network.RetrofitClient;

public class SerieActivity extends AppCompatActivity {

    RecyclerView recyclerView;
    ProgressBar progressBar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_serie);

        int idCat = getIntent().getIntExtra("idCat", 0);
        recyclerView = findViewById(R.id.recyclerDvds);
        progressBar = findViewById(R.id.progressBar);
```



```
recyclerView.setLayoutManager(new
LinearLayoutManager(this));

ApiService api =
RetrofitClient.getInstance().create(ApiService.class);

api.getDvdsByCategory(idCat).enqueue(new
Callback<DvdResponse>() {

    @Override
    public void onResponse(Call<DvdResponse> call,
                          Response<DvdResponse> response)
    {
        progressBar.setVisibility(View.GONE);
        if (response.isSuccessful() && response.body() !=
null) {
            List<DvdModel> dvds =
response.body().getTable();
            for (DvdModel dvd : dvds) {
                int resId = getResources().getIdentifier(
                    dvd.getImgVideo(), "drawable",
getPackageName());
                if (resId != 0) dvd.setImgResId(resId);
            }
            recyclerView.setAdapter(
                new DvdRecyclerViewAdapter(SerieActivity.this,
dvds));
            recyclerView.setVisibility(View.VISIBLE);
        } else {
            Toast.makeText(SerieActivity.this,
                "Réponse vide ou erreur serveur",
                Toast.LENGTH_LONG).show();
        }
    }

    @Override
    public void onFailure(Call<DvdResponse> call, Throwable
t) {
        progressBar.setVisibility(View.GONE);
        Toast.makeText(SerieActivity.this,
            "Erreur réseau : " + t.getMessage(),
            Toast.LENGTH_LONG).show();
    }
});
}
```



10 Tester l'application

Checklist avant de lancer le projet :

- ☐ Le serveur PHP local est démarré (WAMP / XAMPP / Laragon)
- ☐ L'API répond bien dans un navigateur :
`http://localhost/api_dvd/server/listByCategory.php?idCat=1`
- ☐ Les images sont placées dans `res/drawable/` avec les bons noms (sans extension, en minuscules)
- ☐ L'émulateur Android est lancé dans Android Studio
- ☐ Cliquer sur Run ► dans Android Studio

 *Si les images ne s'affichent pas : le nom dans la base de données ne correspond probablement pas au nom du fichier dans drawable. Les noms doivent être identiques, en minuscules, sans extension.*

 *Si l'appli est testée depuis un vrai téléphone (pas l'émulateur) : remplacer 10.0.2.2 par l'adresse IP de la machine sur le réseau Wi-Fi (ex: 192.168.1.x) dans `RetrofitClient.java`.*